

Guía Técnica: Optimización con Apache Parquet y PySpark

Esta guía detalla el flujo de trabajo profesional para la gestión de datos a gran escala, desde la ingesta de archivos planos hasta la persistencia optimizada en formatos columnares.

1. ¿Por qué Parquet es el estándar en Data Engineering?

A diferencia de los formatos de texto como CSV o JSON, Parquet es un formato **columnar** que ofrece ventajas críticas para el procesamiento distribuido:

- **Compresión superior:** Al agrupar datos del mismo tipo, los algoritmos de compresión son mucho más eficientes, ahorrando hasta un 75% de espacio en disco.
- **Proyección de columnas (Column Projection):** Spark solo lee las columnas necesarias para una consulta, ignorando el resto. Esto reduce drásticamente el I/O (entrada/salida) de disco.
- **Preservación de Esquema:** El archivo contiene metadatos con los tipos de datos (int, string, timestamp), eliminando la necesidad de inferir el esquema en cada lectura.

2. El concepto de Particionamiento Físico

Tener un solo archivo gigante es un antipatrón en Big Data. La división en múltiples archivos (particiones) es mejor por los siguientes motivos:

- **Paralelismo Real:** Spark asigna una tarea a cada núcleo (core) disponible para leer un archivo. Si tienes 4 archivos, 4 workers pueden trabajar simultáneamente. Si tienes 1 solo, solo 1 worker trabaja mientras los demás esperan.
- **Escalabilidad:** Facilita el manejo de datasets que superan la capacidad de memoria de un solo nodo.
- **Recuperación de fallos:** Si un proceso falla mientras escribe, es más fácil reintentar la escritura de una partición pequeña que la de un archivo de varios Terabytes.

3. Implementación en PySpark (Código Maestro)

A continuación, se detalla el flujo completo para convertir un CSV, particionarlo y leerlo de forma unificada.

```
# 1. CARGA DE DATOS CRUDOS
# Leemos el CSV original (formato lento e ineficiente)
df = spark.read.csv("datos_origen.csv", header=True,
inferSchema=True)

# 2. TRANSFORMACIÓN Y REPARTICIÓN
# Forzamos la división en 4 archivos para maximizar el paralelismo
del clúster
```

```
# Luego guardamos en formato Parquet en una carpeta contenedora
df.repartition(4).write.mode("overwrite").parquet("data_storage/mi_d
ataset_optimizado")

# 3. LECTURA DE CARPETA UNIFICADA
# Spark trata la CARPETA como una sola tabla, uniendo los archivos
internos automáticamente
df_final = spark.read.parquet("data_storage/mi_dataset_optimizado")

# 4. ACCIÓN PARA VERIFICAR
print(f"Total de registros procesados: {df_final.count()}")
df_final.show(5)
```

4. Estructura de Archivos Resultante

Al ejecutar el código anterior, en tu sistema de archivos verás una estructura similar a esta:

```
mi_dataset_optimizado/
├─ _SUCCESS
├─ part-00000-xxxx.snappy.parquet
├─ part-00001-xxxx.snappy.parquet
├─ part-00002-xxxx.snappy.parquet
└─ part-00003-xxxx.snappy.parquet
```

Nota: El archivo `_SUCCESS` es vital, ya que indica que la operación de escritura terminó correctamente en todos los nodos.